

Lab 3: Encapsulation and Inheritance

1. Encapsulation

Encapsulation is one of the four fundamental OOP principles (Encapsulation, Inheritance, Polymorphism, and Abstraction). It restricts direct access to an object's internal state and ensures that modifications are done only through controlled methods (getters and setters).

Key Benefits of Encapsulation:

- Hides implementation details from the user.
- Protects the integrity of the object's data.
- Makes the code more maintainable and flexible.

```
class BankAccount {  
    private double balance; // Private variable, cannot be accessed directly  
  
    // Constructor to initialize balance  
  
    public BankAccount(double balance) {  
        this.balance = balance;  
    }  
  
    // Getter method to retrieve balance  
  
    public double getBalance() {  
        return balance;  
    }  
}
```

```

// Setter method to modify balance (ensures valid balance)

public void deposit(double amount) {
    if (amount > 0) {
        balance += amount;
        System.out.println("Deposited: " + amount);
    } else {
        System.out.println("Invalid deposit amount!");
    }
}

// Withdrawal method (encapsulating logic to prevent overdrawing)

public void withdraw(double amount) {
    if (amount > 0 && amount <= balance) {
        balance -= amount;
        System.out.println("Withdrawn: " + amount);
    } else {
        System.out.println("Insufficient balance or invalid amount!");
    }
}
}

```

```

public class Main {
    public static void main(String[] args) {
        BankAccount account = new BankAccount( balance: 1000);

        // Using encapsulated methods to access data
        System.out.println("Balance: " + account.getBalance());

        account.deposit( amount: 500);
        account.withdraw( amount: 300);

        // System.out.println(account.balance); // ERROR: 'balance' is private
    }
}

```

Access Modifiers

1. **Public** – The member is accessible from anywhere in the program.
2. **Private** – The member is only accessible within the same class.
3. **Protected** – The member is accessible within the same package and by subclasses.

```
class Animal {  
    public String name = "Lion"; // Accessible everywhere  
    private int age = 5;         // Accessible only in this class  
    protected String type = "Mammal"; // Accessible in subclass and same package  
    String habitat = "Savanna"; // Default (package-private), accessible within package  
  
    private void privateMethod() {  
        System.out.println("This is a private method.");  
    }  
  
    public void showDetails() {  
        System.out.println("Animal: " + name + ", Age: " + age + ", Type: " + type);  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Animal a = new Animal();  
        System.out.println(a.name); // Allowed (public)  
        // System.out.println(a.age); // ERROR: age is private  
    }  
}
```

Modifier	Within Class	Within Package	Outside Package (Subclass)	Outside Package (Non-Subclass)
private	✓	✗	✗	✗
default (no modifier)	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

Getter & Setter Methods

Getters and setters allow controlled access to private variables.

```
class Student {  
    private String name;  
    private int age;  
  
    // Getter method for name  
    public String getName() {  
        return name;  
    }  
  
    // Setter method for name  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    // Getter method for age  
    public int getAge() {  
        return age;  
    }  
  
    // Setter method for age with validation  
    public void setAge(int age) {  
        if (age > 0) {  
            this.age = age;  
        } else {  
            System.out.println("Age must be positive!");  
        }  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Student student = new Student();  
        student.setName("Alice");  
        student.setAge(20);  
  
        System.out.println("Student Name: " + student.getName());  
        System.out.println("Student Age: " + student.getAge());  
    }  
}
```

Static vs Non-Static

Static

- Belongs to the **class** rather than an instance.
- Shared among all instances.
- Can be accessed without creating an object.

Non-Static

- Belongs to an **instance** of the class.
- Requires object creation to access.

Example: Static vs Non-Static

```
class MathOperations {
    static int count = 0; // Static variable (shared among all instances)

    // Static method (can be accessed without an object)
    public static int add(int a, int b) {
        return a + b;
    }

    // Non-static variable (different for each instance)
    int instanceVariable = 10;

    // Non-static method (requires an object)
    public void showInstanceVariable() {
        System.out.println("Instance Variable: " + instanceVariable);
    }
}
```

```

public class Main {
    public static void main(String[] args) {
        // Calling static method without an object
        System.out.println("Sum: " + MathOperations.add(a: 5, b: 10));

        // Using static variable
        MathOperations.count++;
        System.out.println("Count: " + MathOperations.count);

        // Creating objects for non-static members
        MathOperations obj1 = new MathOperations();
        MathOperations obj2 = new MathOperations();
        obj1.instanceVariable = 20; // Each instance has its own variable

        obj1.showInstanceVariable(); // Outputs: 20
        obj2.showInstanceVariable(); // Outputs: 10
    }
}

```

2. Inheritance

Inheritance is an Object-Oriented Programming (OOP) concept where one class (child/subclass) inherits properties and behaviors (fields and methods) from another class (parent/superclass).

Key Benefits of Inheritance:

- Code reusability (avoid redundancy).
- Improves maintainability.
- Supports hierarchical classification (IS-A relationship).

```

class Parent { // Superclass (Base Class)
    void display() {
        System.out.println("This is a method from Parent class.");
    }
}

class Child extends Parent { // Subclass (Derived Class)
    void show() {
        System.out.println("This is a method from Child class.");
    }
}

public class Main {
    public static void main(String[] args) {
        Child obj = new Child();
        obj.display(); // Inherited from Parent
        obj.show();    // Defined in Child class
    }
}

```

Single Inheritance

- A single child class inherits from a single parent class.
- Simplest form of inheritance.

```

class Animal {
    void eat() {
        System.out.println("This animal eats food.");
    }
}

class Dog extends Animal { // Dog inherits from Animal
    void bark() {
        System.out.println("The dog barks.");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.eat(); // Inherited method from Animal
        dog.bark(); // Method of Dog class
    }
}

```

Multilevel Inheritance

- A class is derived from another derived class.
- Forms a chain of inheritance.

```
class Grandparent {  
    void greet() {  
        System.out.println("Hello from Grandparent!");  
    }  
}  
  
1 inheritor  
class Parent extends Grandparent {  
    void sayHello() {  
        System.out.println("Hello from Parent!");  
    }  
}  
  
class Child extends Parent {  
    void introduce() {  
        System.out.println("Hello from Child!");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Child obj = new Child();  
        obj.greet();    // Inherited from Grandparent  
        obj.sayHello(); // Inherited from Parent  
        obj.introduce(); // Defined in Child  
    }  
}
```


Hierarchical Inheritance

- A single parent class has multiple child classes.
- Each child class inherits properties from the same parent class but does not share properties with other child classes.

```
class Vehicle {
    void start() {
        System.out.println("Vehicle is starting...");
    }
}

class Car extends Vehicle {
    void drive() {
        System.out.println("Car is driving...");
    }
}

class Bike extends Vehicle {
    void ride() {
        System.out.println("Bike is riding...");
    }
}

public class Main {
    public static void main(String[] args) {
        Car car = new Car();
        car.start(); // Inherited from Vehicle
        car.drive(); // Defined in Car

        Bike bike = new Bike();
        bike.start(); // Inherited from Vehicle
        bike.ride(); // Defined in Bike
    }
}
```

Note: Java does NOT support Multiple Inheritance (where a class inherits from more than one parent class) due to ambiguity issues. Instead, Java provides Interfaces to achieve multiple inheritance.

Exercise

1. University Student Graduation Management

Assume that the University of Information Technology in Ho Chi Minh City trains students in two programs: College and University.

The information required for a **College student** includes:

- Student ID
- Full name
- Address
- Total credits
- GPA
- Graduation exam score

The information required for a **University student** includes:

- Student ID
- Full name
- Address
- Total credits
- GPA
- Thesis title
- Thesis score

Graduation Criteria:

- College students graduate if they have at least 120 total credits, a GPA of 5.0 or higher, and a graduation exam score of 5.0 or higher.
- University students graduate if they have at least 170 total credits, a GPA of 5.0 or higher, and a thesis score of 5.0 or higher.

Requirements:

Write a Java program that:

- Allows input of a list of students (strings can be used for text fields).
- Determines the number of students who meet graduation requirements.
- Finds the University student with the highest GPA.

2. Ticket Management for Dam Sen Park

During the summer, Dam Sen Park welcomes children, students, and visitors to enjoy various games.

Each **ticket** contains:

- Ticket ID (String)
- Holder's full name (String)
- Year of birth (Integer)
- Number of games played (Integer)

Ticket Types:

1. Full Package Ticket

- Price: 200,000 VND
- Allows access to all 30 games in the park.

2. Partial Ticket

- Base price: 70,000 VND (entry fee)
- Each game costs an additional 20,000 VND

Requirements:

Write a Java program that:

- Inputs a list of tickets.
- Calculates the total revenue from ticket sales.
- Determines the number of Partial Tickets sold.

3. Hotel Room Rental Management

Room Types & Prices:

- VIP1: 900k VND/day
- VIP2: 700k VND/day
- VIP3: 500k VND/day

Information to Manage:

1. Guest:

- ID Card (CCCD)
- Full Name
- Age

2. Hotel Staff:

- License ID
- Employee ID
- Full Name
- Age

Requirements:

1. Create a base class **Person** to manage both hotel guests and staff.
2. Create derived classes for **Guest** and **Staff**.
3. Create a **Room** class to manage room details.
4. Implement methods to:
 - Add or remove guests.
 - Calculate rental cost:

$$\text{Total Cost} = \text{Room Price} \times \text{Number of Days}$$

(Number of days is inputted by the user).

4. Singer Salary Management for XYZ Company

XYZ Company manages **singers' salary calculation**. Singers are categorized into:

1. Regular Singers
2. Famous Singers

Common Information:

- Full name
- Years of experience in the company
- Number of albums sold
- Number of performances

Additional Information for Famous Singers:

- Number of game shows attended

Salary Calculation:

- **Regular Singers:**

$\text{Salary} = 3,000,000 + 500,000 \times \text{years} + 1,000 \times \text{albums sold} + 200,000 \times \text{performances}$

- **Famous Singers:**

$\text{Salary} = 5,000,000 + 500,000 \times \text{years} + 1,200 \times \text{albums sold} + 500,000 \times \text{performances} + 500,000 \times \text{game shows}$

Requirements:

Write a **Java program** that:

1. Inputs a list of singers (stored in a single array).
2. Finds the highest-paid singer.
 - If multiple singers have the highest salary, return any one of them.

Constraints:

- Use Inheritance
- Use Strings for text fields

- Clearly define and implement all classes

5. Cosmetic Store Order Management

A **cosmetic store** wants to manage **sold orders** through a software system.

Order Information:

- Order ID
- Customer information
- Invoice date
- Order price (based on package price)
- Product package information

Customer Information:

- Customer ID
- Full name
- Address
- Phone number

Product Packages:

Two types of **predefined packages**:

1. Romantic Package

- Always includes Rose Perfume
- Shampoo & shower gel are optional

2. Fresh-Air Package

- Always includes Oily Hair Shampoo
- Perfume & shower gel are optional

Product Details:

1. Perfume (Types: Rose, Chamomile)

- Attributes: ID, Type, Volume (ml), Price
- Price Calculation:
 - Rose Perfume = Volume $\times$ 20,000 VND/ml
 - Chamomile Perfume = Volume $\times$ 15,000 VND/ml

2. Shampoo (Types: For Dry Hair, For Oily Hair)

- Attributes: ID, Type, Volume (ml), Price
- Additional Certification for Oily Hair Shampoo:
 - Standard 1: 30,000 VND/ml
 - Standard 2: 40,000 VND/ml
- Price Calculation:
 - Dry Hair Shampoo = Volume $\times$ 20,000 VND/ml

3. Shower Gel (Types: For Oily Skin, For Dry Skin)

- Attributes: ID, Type, Volume (ml), Price
- Price Calculation:
 - Dry Skin Shower Gel = Volume $\times$ 20,000 VND/ml
 - Oily Skin Shower Gel = Volume $\times$ 40,000 VND/ml

Requirements:

Write a **Java program** that:

1. Inputs N orders
2. Displays all orders before saving
3. Saves the order list to a file (order_list.txt)